Computer Vision and Image Processing Techniques in MATLAB

Author: Justin Ogle

Project Overview

This project demonstrates foundational computer vision and image processing techniques implemented in MATLAB. The objective is to explore image preprocessing, edge detection, image enhancement, convolution-based filtering, and deep learning image classification using industry-standard methodologies.

The project combines classical image processing approaches with modern deep learning techniques to illustrate how images can be analyzed, enhanced, and classified. These methods form the basis of many applications in computer vision, robotics, autonomous systems, medical imaging, and artificial intelligence.

Techniques Demonstrated

Image Resizing and Preprocessing

Images are resized and prepared for subsequent processing and analysis. Standardized image dimensions are commonly used for machine learning and computer vision workflows.

Edge Detection Using Convolution Filters

The Sobel and Prewitt operators are implemented using convolution to detect image boundaries and structural features. The results are compared to evaluate filter behavior and edge detection performance.

Deep Learning Image Classification

A pre-trained GoogLeNet convolutional neural network is used for real-time image classification through a webcam interface. Classification confidence scores and top prediction probabilities are displayed to evaluate model performance.

Image Enhancement and Feature Extraction

A multi-stage enhancement pipeline is implemented using Laplacian filtering, Sobel edge detection, averaging filters, image masking, and gamma correction. The resulting workflow improves image sharpness and visual feature representation.

Applications

The techniques demonstrated in this project are foundational to:

- Computer Vision Systems
- Object Detection and Recognition
- Autonomous Vehicles
- Robotics and Automation
- Medical Image Analysis
- Image Analytics
- Artificial Intelligence and Machine Learning

Software and Tools

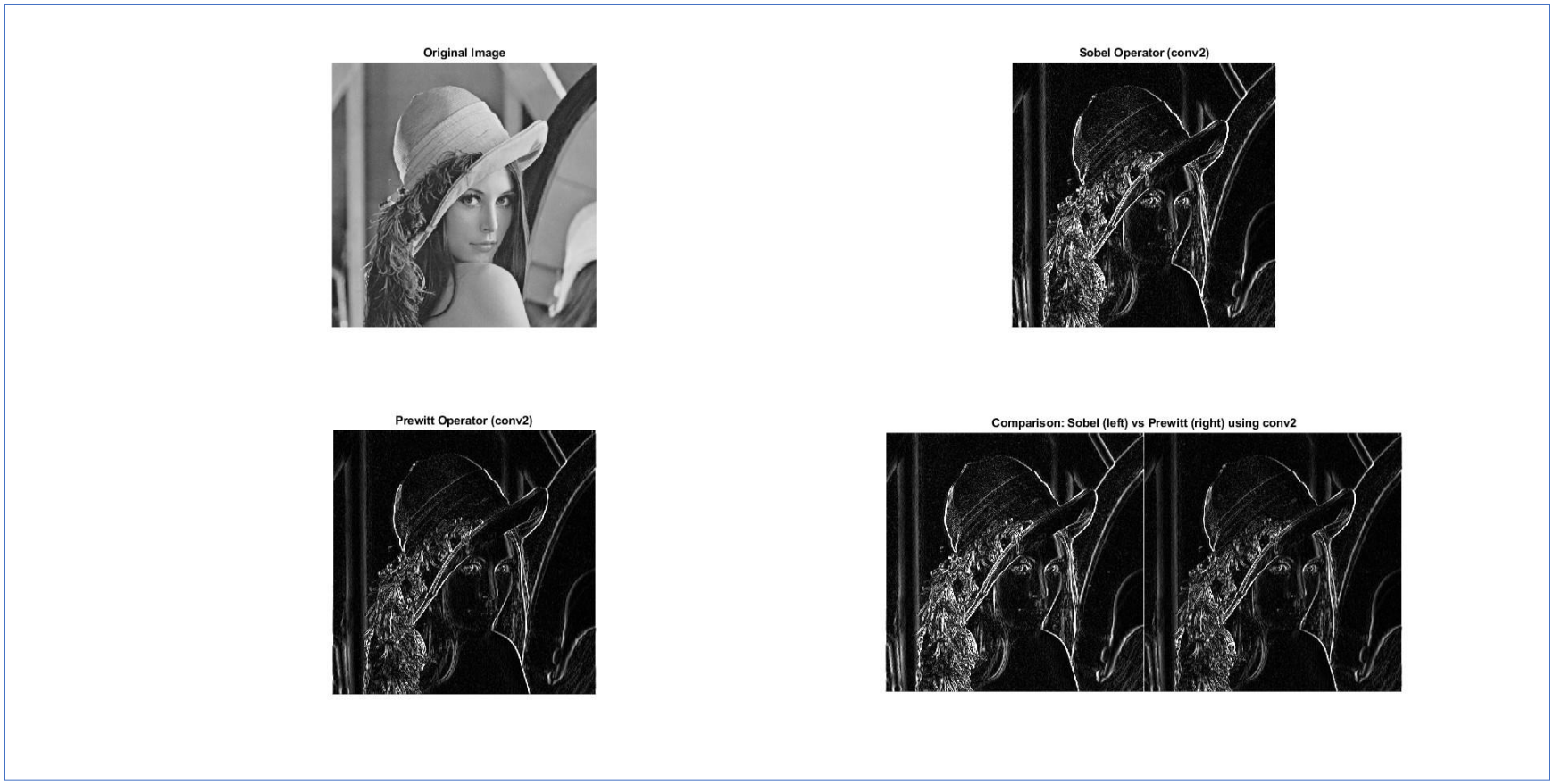
- MATLAB
- Deep Learning Toolbox
- Image Processing Toolbox
- GoogLeNet Pretrained Network
- Webcam Support Package

Conclusion

This project provides practical experience with both traditional image processing algorithms and modern deep learning techniques. Together, these approaches form a foundation for developing advanced computer vision and artificial intelligence systems.

```
% Image Resizing and Preprocessing
% Load image
I = imread('Lena.png');

% Resize image to 224 x 224 pixels
resized_image = imresize(I,[224 224]);

% Display original and resized images
figure;
subplot(1,2,1);
imshow(I);
title('Original Lena Image');

subplot(1,2,2);
imshow(resized_image);
title('Resized Lena Image (224x224)');
```

```
%%%%%%%%%%%%%%%%%%

% Edge Detection Using Convolution Filters
% Load the image
I = imread('Lena.png');

% Convert the image to grayscale
G = rgb2gray(I);

% Display the original grayscale image
figure;
subplot(2, 2, 1), imshow(G), title('Original Image');

% Sobel edge operator using conv2
sx_sobel = [-1, 0, 1; -2, 0, 2; -1, 0, 1];
CG_sobel = conv2(double(G), sx_sobel, 'same');

% Display the result of Sobel operator using conv2
subplot(2, 2, 2), imshow(uint8(abs(CG_sobel))), title('Sobel Operator (conv2)');

% Prewitt edge operator using conv2
sx_prewitt = [-1, 0, 1; -1, 0, 1; -1, 0, 1];
CG_prewitt = conv2(double(G), sx_prewitt, 'same');

% Display the result of Prewitt operator using conv2
subplot(2, 2, 3), imshow(uint8(abs(CG_prewitt))), title('Prewitt Operator (conv2)');

% Display both Sobel and Prewitt results side by side using montage
subplot(2, 2, 4), imshowpair(uint8(abs(CG_sobel)), uint8(abs(CG_prewitt)), 'montage');
title('Comparison: Sobel (left) vs Prewitt (right) using conv2');
```




```
%%%%%%%%%%%%%%%%%%

% Deep Learning Image Classification Using GoogLeNet
% https://www.mathworks.com/help/deeplearning/ug/classify-images-from-webcam-using-deep-learning.html

% Initialize the webcam
camera = webcam;

% Load the pre-trained GoogLeNet model
net = googlenet;
```

```
% Get the input size required by the first layer of the network
inputSize = net.Layers(1).InputSize(1:2);

% Display the input size
disp('Network Input Size:');
disp(inputSize);
```

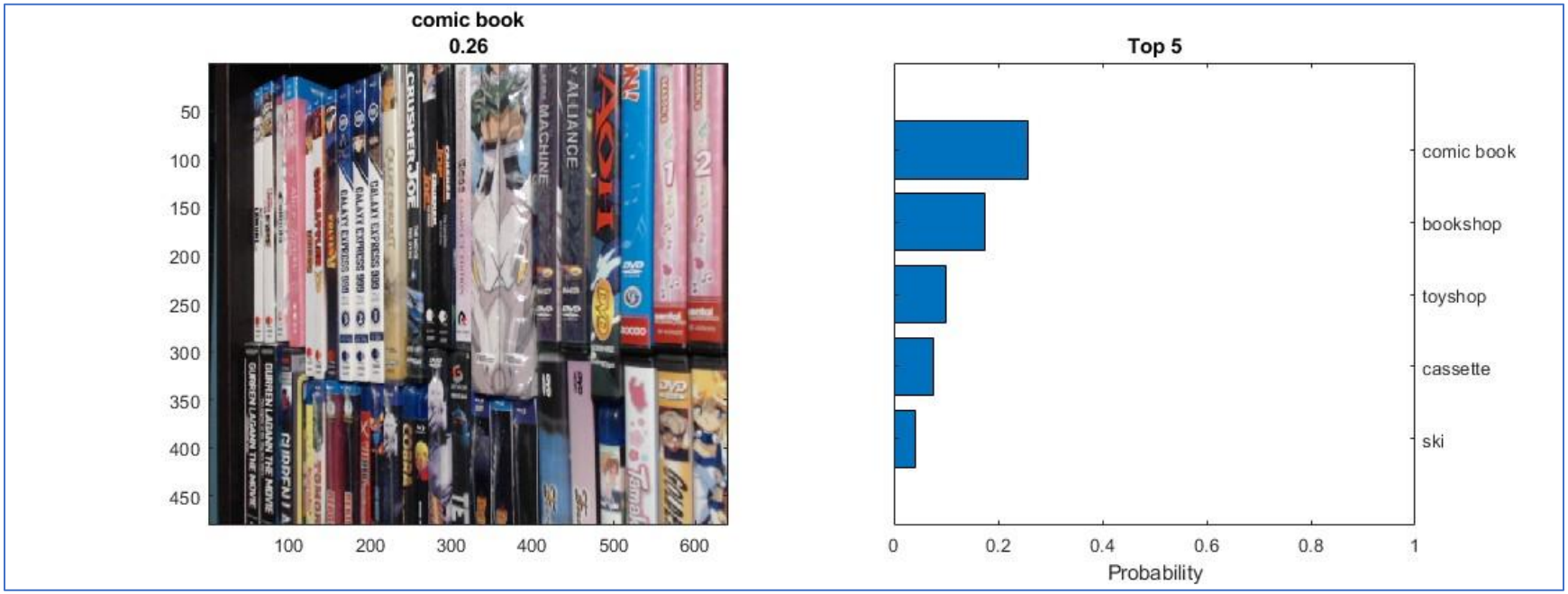

```
% Create a figure for displaying the initial webcam image and its classification
figure;
im = snapshot(camera);
```

```
image(im);
im = imresize(im, inputSize);
[label, score] = classify(net, im);
title({char(label), num2str(max(score), 2)});

% Create a figure for continuous webcam streaming and classification
h = figure;

% Continuously update the figure while it is open
while ishandle(h)
    % Capture an image from the webcam
    im = snapshot(camera);

    % Display the captured image
    image(im);

    % Resize the image to the required input size for the network
    im = imresize(im, inputSize);

    % Classify the image using the pre-trained GoogLeNet model
    [label, score] = classify(net, im);

    % Display the updated classification results as the title of the figure
    title({char(label), num2str(max(score), 2)});

    % Force the figure to update and refresh
    drawnow;
end
```

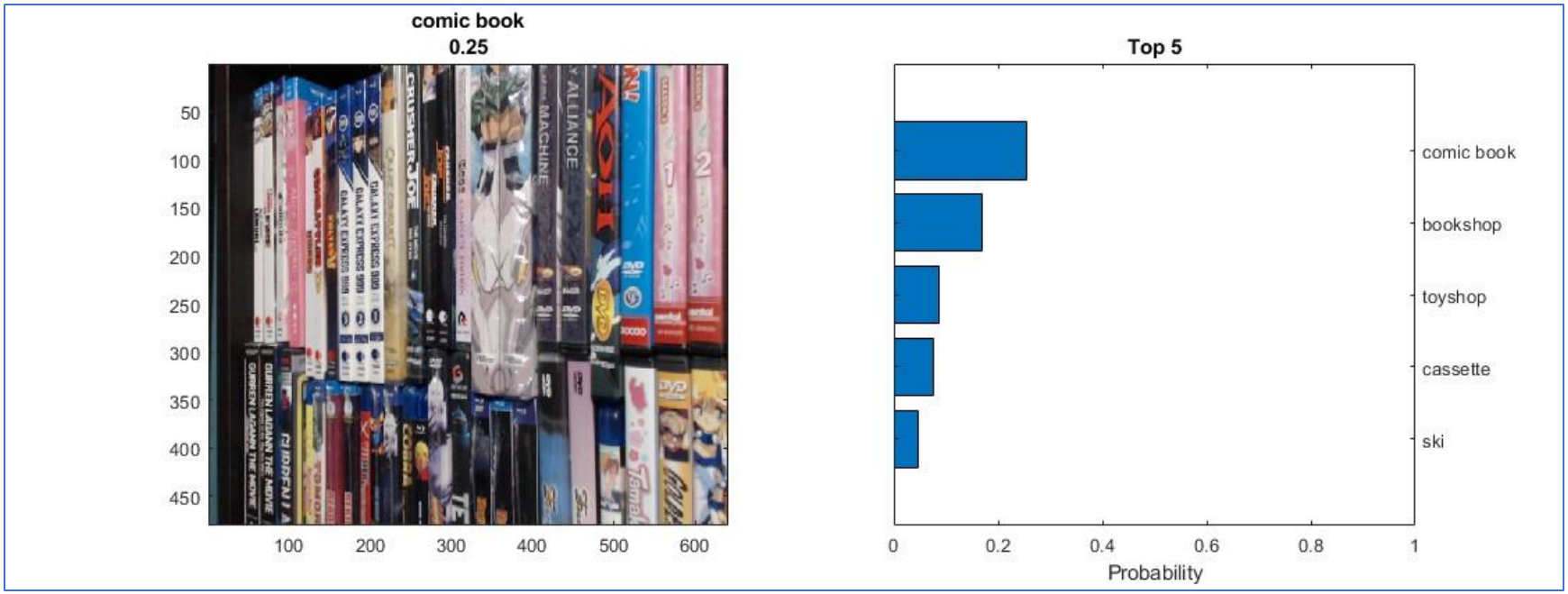

```
% Create a figure with two subplots for displaying webcam images and top 5 probabilities
h = figure;
h.Position(3) = 2 * h.Position(3);
ax1 = subplot(1, 2, 1);
ax2 = subplot(1, 2, 2);

% Capture an image from the webcam
im = snapshot(camera);
image(ax1, im);
im = imresize(im, inputSize);
[label, score] = classify(net, im);
```

```
title(ax1, {char(label), num2str(max(score), 2)});

% Sort the scores in descending order and select the top 5 predictions
[~, idx] = sort(score, 'descend');
idx = idx(5:-1:1);
classes = net.Layers(end).Classes;
classNamesTop = string(classes(idx));
scoreTop = score(idx);

% Plot a horizontal bar chart for the top 5 predictions
barh(ax2, scoreTop);
xlim(ax2, [0 1]);
title(ax2, 'Top 5');
xlabel(ax2, 'Probability');
yticklabels(ax2, classNamesTop);
ax2.YAxisLocation = 'right';

% Create a figure with two subplots for continuous webcam streaming and top 5 probabilities
h = figure;
h.Position(3) = 2 * h.Position(3);
ax1 = subplot(1, 2, 1);
ax2 = subplot(1, 2, 2);
ax2.PositionConstraint = 'innerposition';

% Continuously update the figure while it is open
while ishandle(h)
    % Display and classify the image
    im = snapshot(camera);
    image(ax1, im);
    im = imresize(im, inputSize);
    [label, score] = classify(net, im);
    title(ax1, {char(label), num2str(max(score), 2)});

    % Select the top five predictions
    [~, idx] = sort(score, 'descend');
    idx = idx(5:-1:1);
    scoreTop = score(idx);
    classNamesTop = string(classes(idx));
```

```
    % Plot the histogram
    barh(ax2, scoreTop);
    title(ax2, 'Top 5');
    xlabel(ax2, 'Probability');
    xlim(ax2, [0 1]);
    yticklabels(ax2, classNamesTop);
    ax2.YAxisLocation = 'right';
```



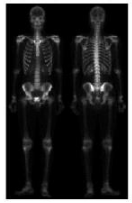

```
    % Force the figure to update and refresh
    drawnow;
end

% Release the webcam when the figure is closed
clear camera;
```




```
% The first display provides continuous real-time webcam classification,
% while the second visualization presents the corresponding prediction
% probabilities for the top five identified classes.

%%%%%%%%%%%%%%%%%%%

% Image Enhancement and Feature Extraction Pipeline
% a.) Show Fig0343(a)(skeleton_orig).tif.
original_image = imread('Fig0343(a)(skeleton_orig).tif');
figure;
subplot(3, 3, 1), imshow(original_image), title('a.) Original Image');

% b.) Take the Laplacian of part a.) then show.
laplacian_image = imfilter(original_image, fspecial('laplacian'));
subplot(3, 3, 2), imshow(laplacian_image), title('b.) Laplacian of Image');

% c.) Sharpen the image of part a.) by adding the Laplacian then show.
```

```
sharpened_image = imadd(original_image, laplacian_image);
subplot(3, 3, 3), imshow(sharpened_image), title('c.) Sharpened Image');

% d.) Take the Sobel of part a.) then show.
sobel_image = edge(original_image, 'sobel');
subplot(3, 3, 4), imshow(sobel_image), title('d.) Sobel of Image');

% e.) Take the Sobel of part a.) and smooth with a 5x5 averaging filter then show.
smoothed_sobel = imfilter(sobel_image, fspecial('average', [5, 5]));
subplot(3, 3, 5), imshow(smoothed_sobel), title('e.) Smoothed Sobel');

% f.) Mask the image formed by the product of part c.) and part e.) then show.
masked_image = immultiply(sharpened_image, smoothed_sobel);
subplot(3, 3, 6), imshow(masked_image), title('f.) Masked Image');

% g.) Sharpen image obtained by the sum of part a.) and part f.) then show.
final_sharpened_image = imadd(original_image, masked_image);
subplot(3, 3, 7), imshow(final_sharpened_image), title('g.) Final Sharpened Image');

% h.) Final result obtained by applying power-law transformation to part g.) then show.
gamma = 2.0;
powerLawTransformedA = imadjust(final_sharpened_image / 255.0, [], [], gamma);
subplot(3, 3, 8), imshow(powerLawTransformedA), title('h.) Power-law Transformed Image');

% i.) Compare part g.) and part h.) with part a.).
subplot(3, 3, 9), imshow([original_image, uint8(final_sharpened_image), im2uint8(powerLawTransformedA)]), title('i.)
Comparison');

% Adjusting the figure for better visualization
set(gcf, 'Position', get(0, 'Screensize'));
```

a.) Original Image

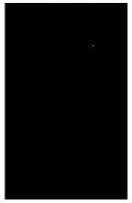


b.) Laplacian of Image

c.) Sharpened Image

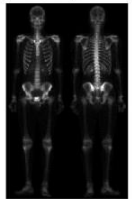
d.) Sobel of Image

e.) Smoothed Sobel



f.) Masked Image

g.) Final Sharpened Image



h.) Power-law Transformed Image

i.) Comparison